

序列模型：RNN 與時間序列預測

循環狀態、LSTM、GRU 與一維卷積

書本第 13 章

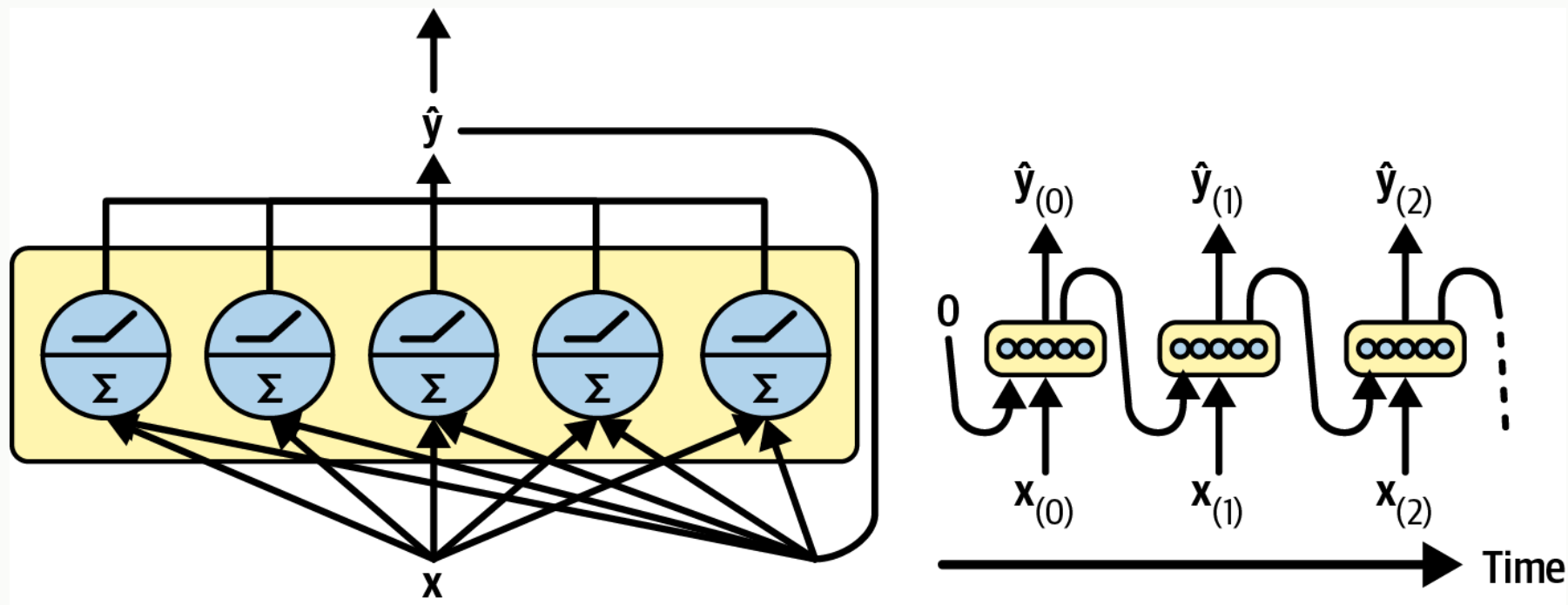
學習重點與成果

- 循環狀態、展開與時間序列基準。
- 資料視窗、RNN 與多步預測。
- LSTM、GRU、因果卷積與實驗設計。

成果：能建立不偷看未來的預測流程，並追蹤序列張量尺寸。

 **配套 Notebook**： `programs/notebooks/16_序列模型_RNN與時間序列預測.ipynb`。本檔提供公式驗算、機制實驗與結果圖；框架片段及完整模型案例另依頁面說明閱讀。

循環層沿時間重用參數



同一組權重處理每個時間步，隱藏狀態攜帶先前資訊。

RNN 的狀態更新

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

x_t ：目前輸入； h_{t-1} ：前一步狀態。

序列較長不會直接增加這層的參數量，卻會增加展開後的計算與記憶體。

循環狀態手算

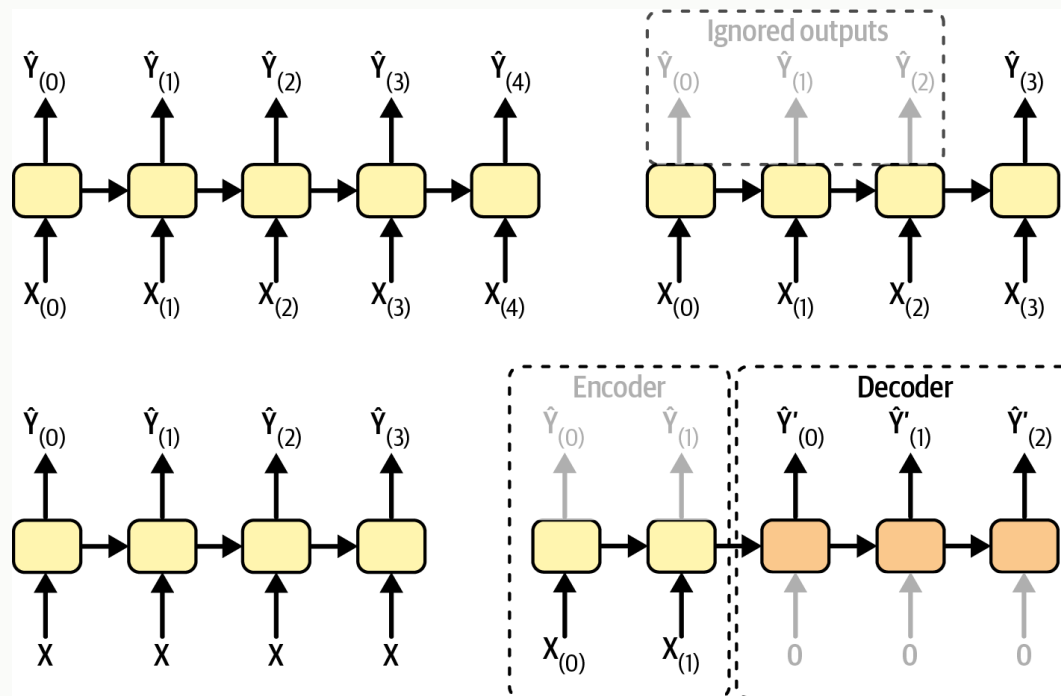
設純量 $h_0 = 0, W_x = 1, W_h = 0.5, b = 0$ ，輸入為 $[1, 0]$ 。

$$h_1 = \tanh(1) \approx 0.762$$

$$h_2 = \tanh(0 + 0.5 \times 0.762) \approx 0.363$$

第二步輸入為 0，為何狀態仍不為 0？

四種序列輸入與輸出

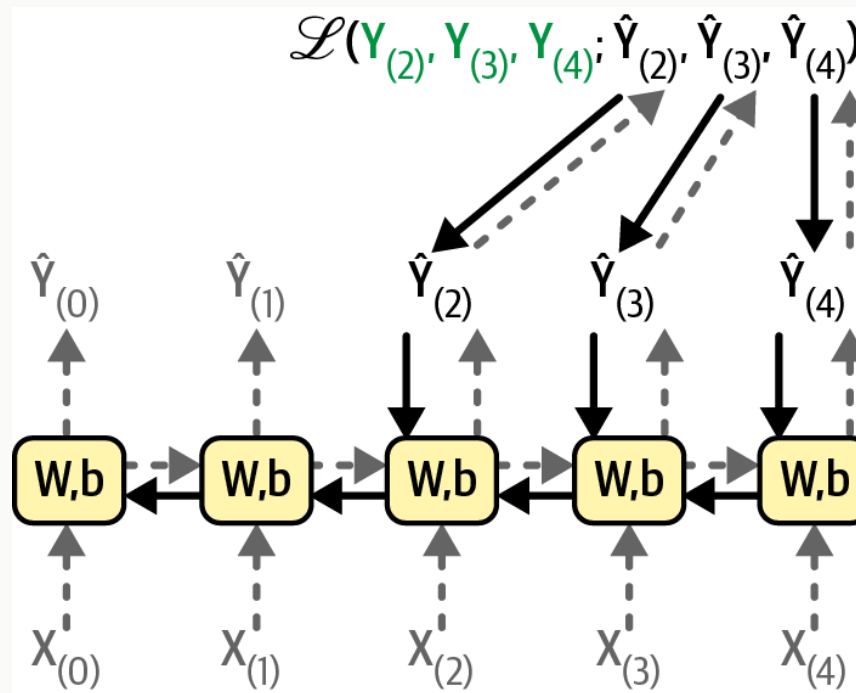


情感分類、逐步預測、音樂生成與翻譯，需要不同的輸出對齊方式。

Hidden state 與 output

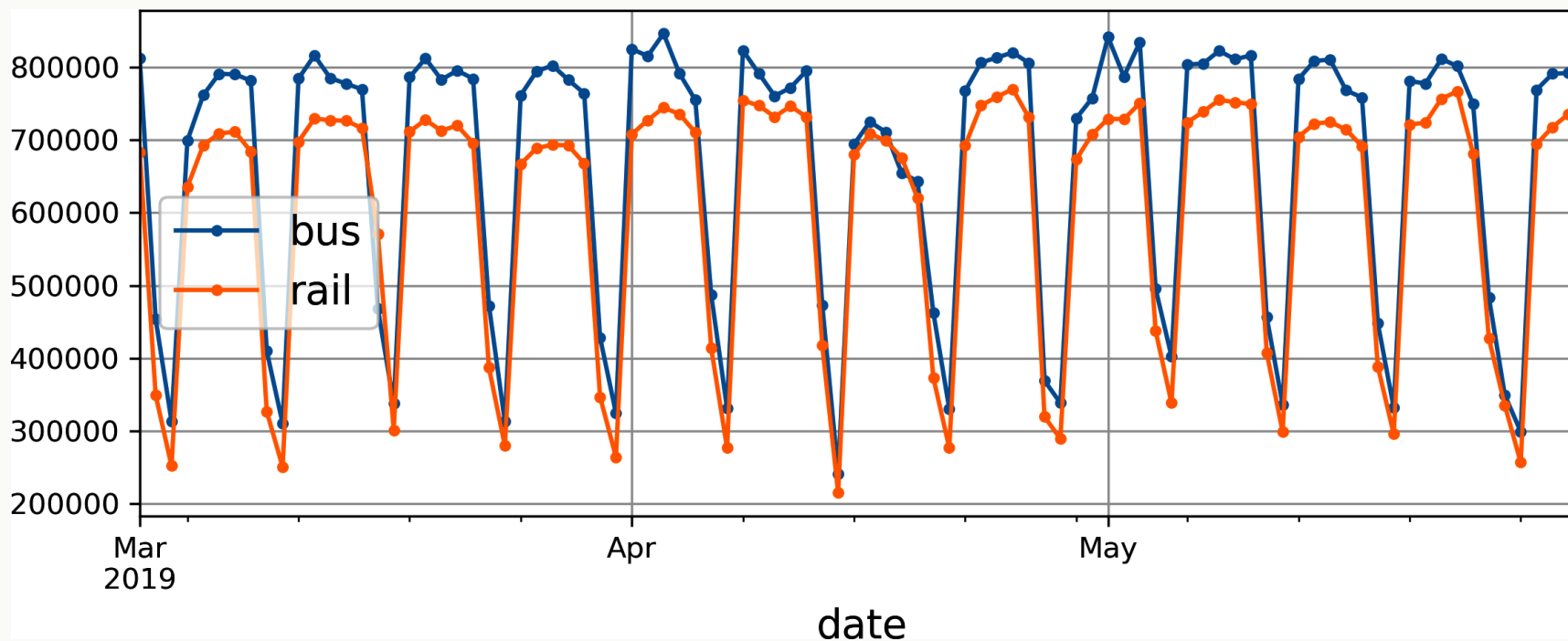
- 簡單 RNN 可把隱藏狀態直接當逐步輸出。
- LSTM 同時維持 hidden state 與 cell state。
- sequence-to-vector 常取最後有效時間步，再接任務頭。
- 變長序列的「最後一步」要排除 padding。

Backpropagation through time



把時間步展開成計算圖；同一參數收到不同時間步的梯度並加總。

時間序列的季節性



芝加哥運量有日期相關的規律；模型應與簡單的週期性基準比較。

先建立樸素基準

方法	明天的預測
Persistence	今天的值
季節性 naive	上週同一天的值
移動平均	最近一段時間平均

畫原始序列、落後 7 天的序列與差值，再決定模型需要學什麼。

ARMA、ARIMA 與 SARIMA

成分	作用
AR	使用過去的觀測值
MA	使用過去的預測誤差
I	以差分處理部分非平穩性
Seasonal	描述固定週期的結構

這裡 MA 指移動平均模型的誤差項，不等於直接做滑動平均平滑。

時間切分與洩漏

先依時間切出訓練、驗證與測試，再建立可使用的歷史視窗。

- 標準化只用訓練期估計統計量。
- 預測當下無法知道的特徵，不可出現在輸入。
- 回測每個時間點，只使用當時已取得的資料。

驗證窗可以引用邊界前已知的歷史，但標籤必須落在驗證期。

單步預測的滑動視窗

```
def make_window(series, i, window_length):  
    end = i + window_length  
    X = series[i:end]  
    y = series[end]  
    return X, y  
# 例: series 有 10 筆, window_length=4, 可建 6 筆樣本
```

輸入 [時間, 特徵] ; DataLoader 加上 batch 軸後為 [B,T,F] 。

線性基準與公平比較

把過去 56 天攤平後接一個線性層，也能產生預測。

比較時固定：資料區間、輸入視窗、預測範圍、評估單位。

若原始資料除以一百萬，MAE 也在縮放後單位；報告時換回人次。

先預測：persistence 把最後觀測當成下一點，遇到轉折時可能如何落後？線性視窗是否一定更好？

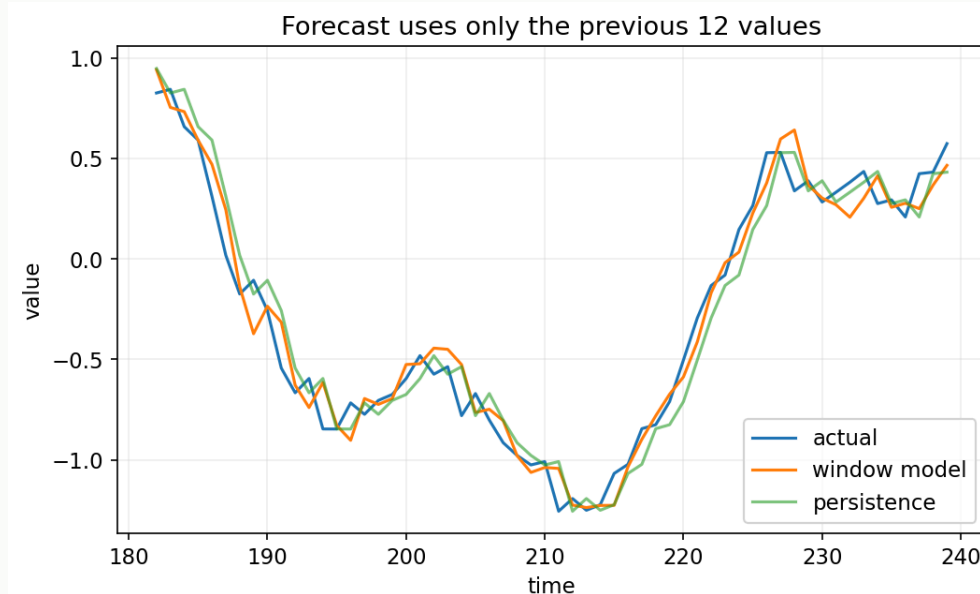
用過去 12 點比較線性視窗與 persistence

合成時間序列先以較早的 170 筆視窗訓練線性迴歸；後段每次只讀當時已知的過去 12 點，預測下一點。

```
pred = model.predict(X[split:])  
persistence = X[split:, -1]
```

橫軸為時間，縱軸為合成數值。藍線是真值，橘線是線性視窗，綠線沿用最後觀測；在轉折附近比較落後情形，再核對 RMSE。

參考程式：[programs/notebooks/16_序列模型_RNN與時間序列預測.ipynb](#)

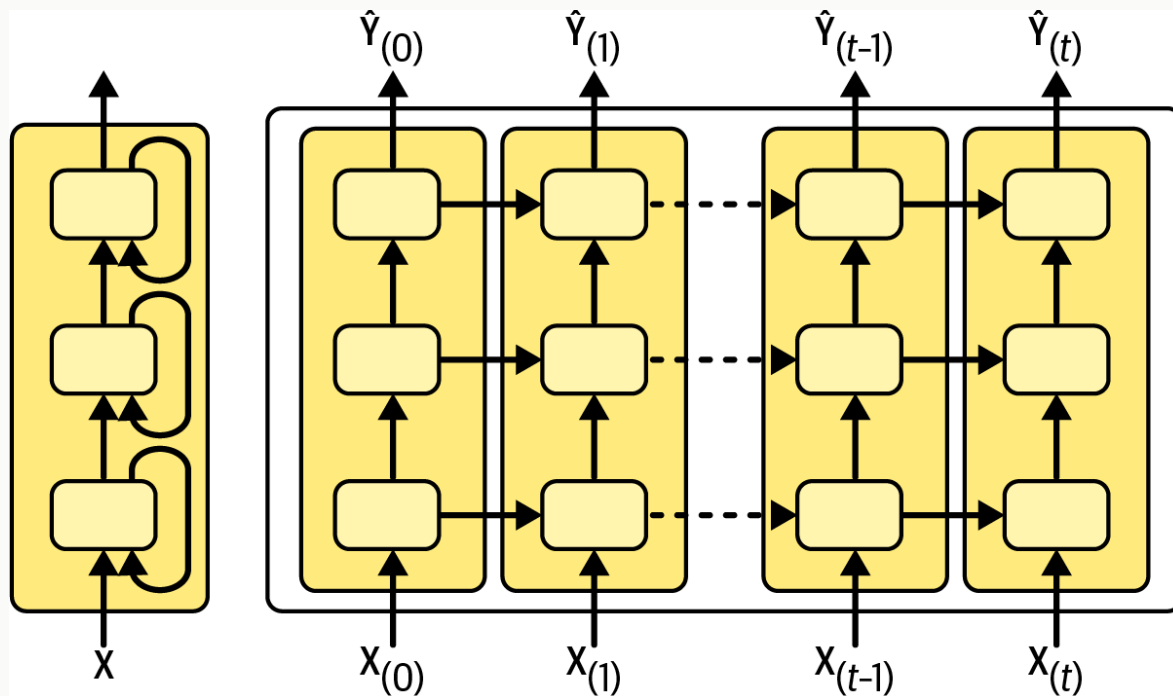


RNN 的張量形狀

```
import torch
from torch import nn
X = torch.randn(8, 56, 1)
rnn = nn.RNN(1, 32, batch_first=True)
outputs, h_n = rnn(X)
head = nn.Linear(32, 1)
y_pred = head(outputs[:, -1])
```

outputs [8,56,32] , h_n [1,8,32] , 預測 [8,1] 。

深層 RNN 的兩種深度



沿時間展開與堆疊循環層是不同方向；增加層數後，每一步的運算也增加。

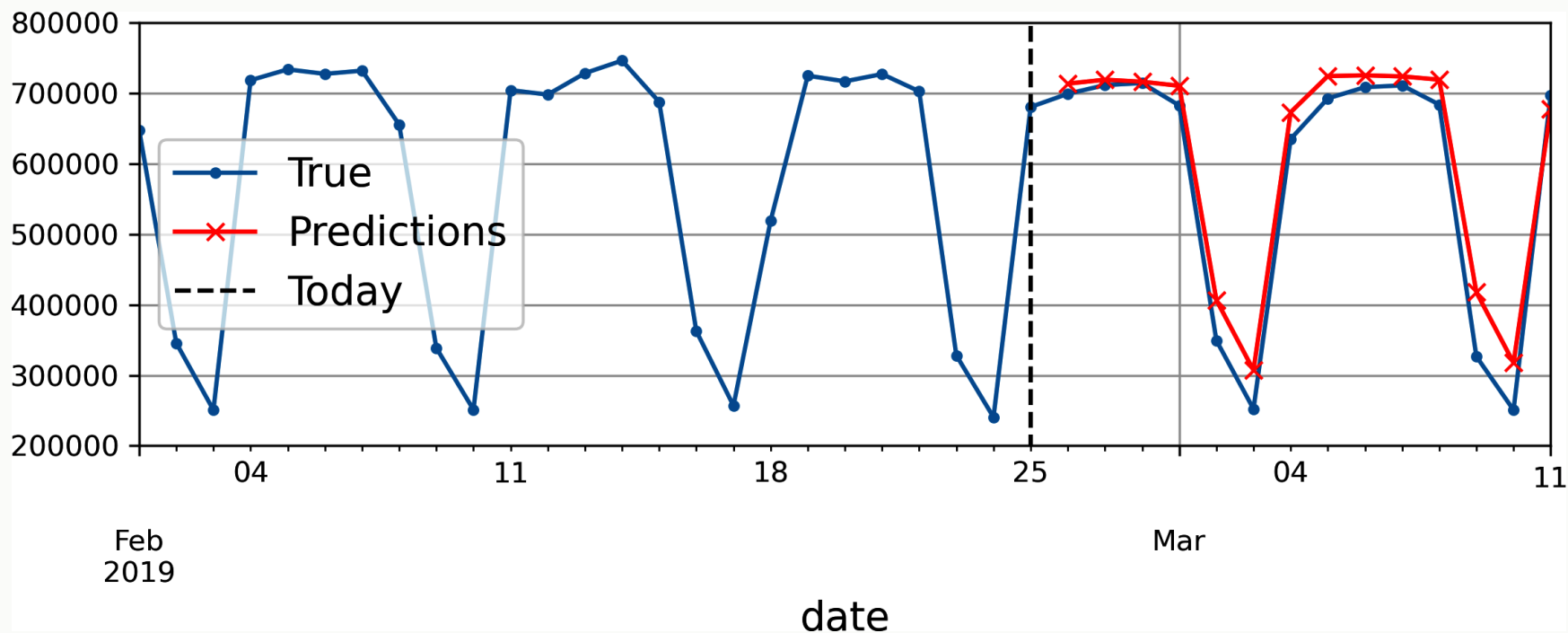
多變量預測的可用性

輸入可以包含公車運量、鐵路運量與日曆資訊。

特徵	預測明日時通常可得？
明天星期幾	可以
明天真實運量	不可以
明天天氣	可用預報，不能偷用事後實測

新增特徵後仍以相同的預測日期做比較。

遞迴式多步預測



先預測下一步，再把預測值放回視窗；誤差可能一路傳播。

多步預測的三種設計

方式	輸出	需要注意
遞迴	反覆產生1步	誤差累積
直接多輸出	一次產生H步	各步的難度不同
Seq2seq	每個時間步都產生未來H步	標籤對齊與因果性

遞迴預測的時間軸

```
# model(X) 輸出 [B,1] ; X 是 [B,T,1]
model.eval()
with torch.no_grad():
    for _ in range(14):
        y_one = model(X)
        X = torch.cat([X, y_one.unsqueeze(1)], dim=1)
predictions = X[:, -14:, 0]
```

`unsqueeze(1)` 加的是時間軸；改成 0 會把 batch 軸弄錯。

Seq2seq 預測的標籤對齊

每個輸入時間步，都輸出接下來 H 步的預測。

輸入位置	當步目標 ($H=3$)
$t=0$	時間1、2、3
$t=1$	時間2、3、4
$t=2$	時間3、4、5

輸出可為 $[B, T, 3]$ ；第 t 步的狀態仍只能使用時間 $\leq t$ 的資訊。

評估多步預測

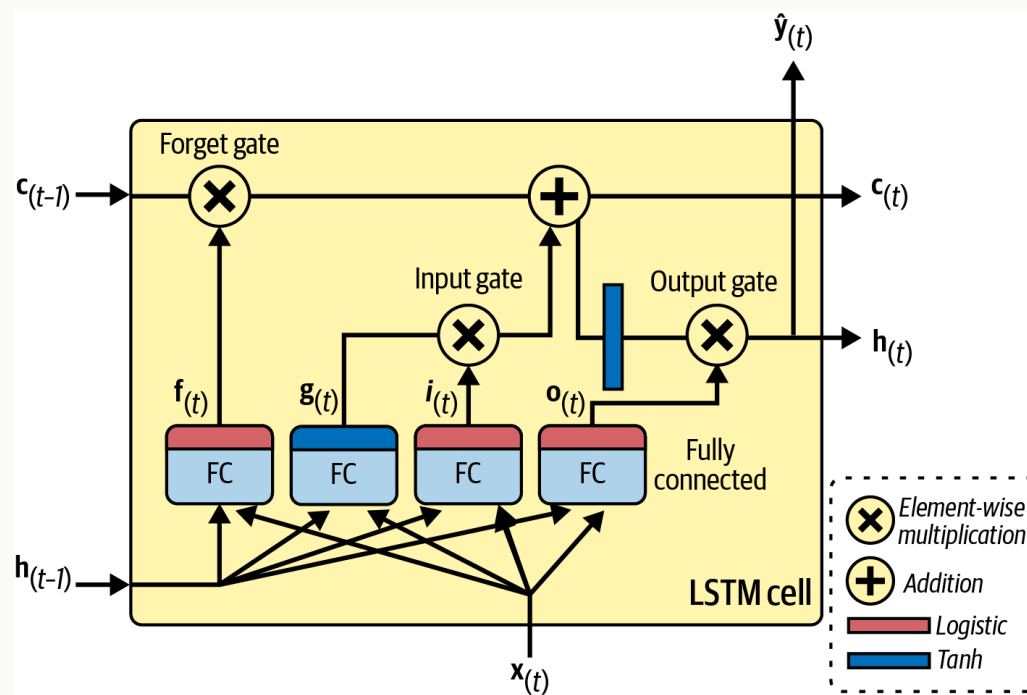
- 分別列出 horizon 1、7、14 的 MAE。
- 同時看逐日誤差與多個回測起點的平均。
- 真實值與預測值必須對齊同一日期。
- 整段平均較低，仍可能掩蓋尖峰或假日失敗。

長序列訓練的困難

梯度反覆穿越時間步，可能消失或爆炸。

- Gradient clipping 限制過大的更新訊號。
- LayerNorm 協助控制每步表示的尺度。
- 截斷 BPTT 限制反向傳播長度，但也限制遠距信用分配。
- 記憶門控針對長期資訊保留提供較好的路徑。

LSTM 的記憶路徑



遺忘門、輸入門、輸出門控制 cell state 如何保留、更新與讀出。

LSTM 的核心更新

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$h_t = o_t \odot \tanh(c_t)$$

門值在 0 到 1 之間；保留與更新可以逐座標調整。

LSTM 能改善長期學習，仍不保證任意長度都能記住。

LSTM 門值手算

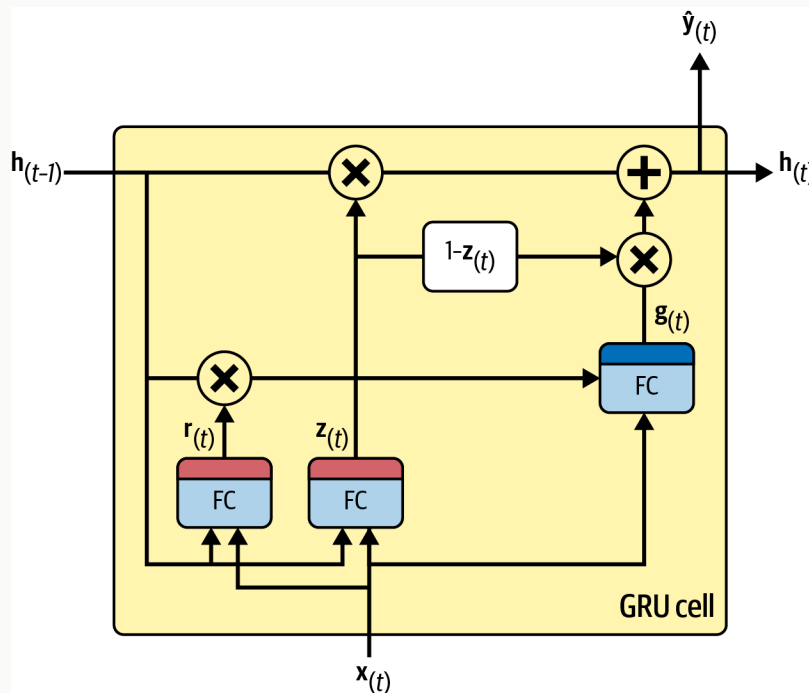
設 $c_{t-1} = 0.8, f_t = 0.9, i_t = 0.2, \tilde{c}_t = -0.5$ 。

$$c_t = 0.9 \times 0.8 + 0.2 \times (-0.5) = 0.62$$

若 $o_t = 0.5$ ， $h_t = 0.5 \tanh(0.62) \approx 0.276$ 。

比較把遺忘門改成 0 時會發生什麼事。

GRU 的精簡門控



GRU 使用更新與重設機制，不另外維持一份獨立 cell state。

RNN、LSTM、GRU 的介面差異

```
lstm = nn.LSTM(3, 32, batch_first=True)
gru = nn.GRU(3, 32, batch_first=True)
X = torch.randn(8, 56, 3)
out_lstm, (h_n, c_n) = lstm(X)
out_gru, h_n_gru = gru(X)
```

LSTM 回傳 hidden 與 cell；兩者逐步輸出皆為 [8, 56, 32]。

一維卷積處理序列

Conv1d 在時間軸套用共享濾波器。

PyTorch 輸入通常為 `[B,C,T]`，與 RNN 的 `[B,T,F]` 不同。

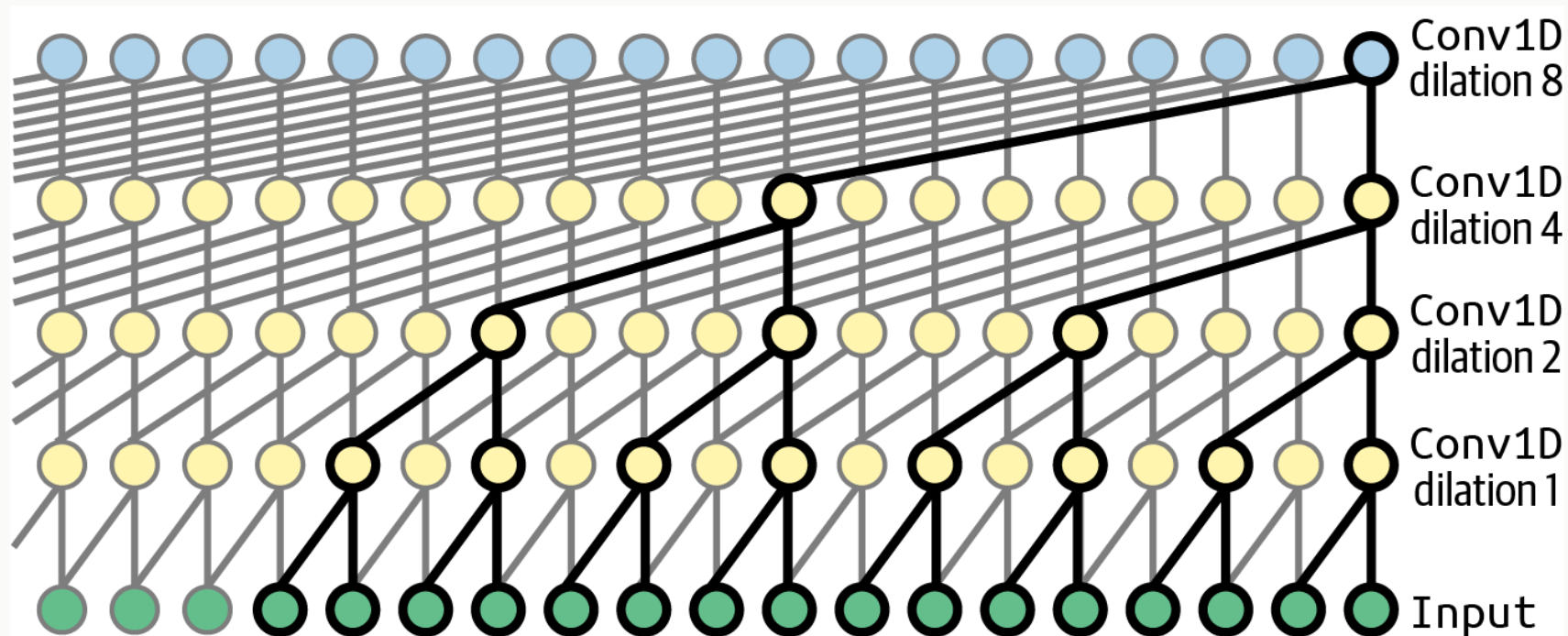
因果預測只能使用現在與過去；置中的 padding 可能讓模型看到未來。

因果卷積只在左側補值

```
import torch.nn.functional as F
class CausalConv1d(nn.Conv1d):
    def forward(self, X):
        left = (self.kernel_size[0] - 1) * self.dilation[0]
        return super().forward(F.pad(X, (left, 0)))
# 建構時 padding=0，避免再做對稱補值
```

用 dilation 增加感受野，仍須維持時間方向。

WaveNet 的擴張卷積



逐層增加 dilation，讓有限層數涵蓋更長歷史；輸出仍維持因果性。

課堂實作：兩週運量預測

先規劃基準模型、時間切分與評估方式；完整訓練可在具備適當算力的環境進行。

1. 建立週期性 naive、線性模型與一個 GRU。
2. 使用同一時間切分，預測未來 14 天。
3. 交付 horizon 誤差表與預測日期對齊示例。
4. 指出一個不應放入模型的未來特徵。

離堂檢核

- hidden state 與模型參數有何不同？
- 為何 RNN 的輸入不能任意打亂時間順序？
- 遞迴預測為何可能越往後越差？
- 因果卷積應在哪一側 padding？

課後程式與延伸閱讀

- 作者第 13 章 Notebook (`13_processing_sequences_using_rnn_and_cnns.ipynb`)：先執行 Setup，再定位本課指定區段。

執行前確認資料、套件與運算資源。

Notebook 導讀：可執行的機制與驗收

實驗	必須核對的條件
RNN 展開、BPTT	狀態手算、共享權重梯度、序列軸
遞迴／direct 多步預測	訓練標籤未跨切分；固定起點不用未來真值
LSTM／GRU	門值與狀態形狀； $c_t = 0.62$
擴張因果卷積	改未來值不改過去輸出；感受野 8

原單步圖是 rolling one-step 評估；多步圖另按 horizon 比較。
合成序列、ARMA 機制與門控前向不等於完成真實運量模型訓練。